

01차시 · LLM 파인튜닝 개발환경 구축

1일차 10:00–11:00 · 60분 실습형 교육자료 · 10페이지
기준: 「2026학년도 보수교육(융합) 과정안내서」, 1일차 1차시
교육 기준: Ubuntu / Python 3.11 / VS Code / Hugging Face / NVIDIA GPU(권장)
작성·공식 문서 확인: 2026-11-04

> 이 차시의 목적은 모델을 학습하는 것이 아니라, 8~11차시의 학습이 가능한 **재현 가능한 실행 환경**을 구축·점검하는 것이다. GPU를 직접 검사하지 않은 상태에서는 학습 가능·불가능을 확정하지 않는다.

1. 60분 운영표

시간	활동	확인할 결과
0~5분	전체 학습 경로와 최종 산출물 이해	12차시 실습 흐름 설명
5~13분	개발환경의 다섯 계층 확인	드라이버/파이썬/패키지/모델/데이터 구분
13~20분	GPU·시스템·디스크 조사	`nvidia-smi`, `df -h` 기록
20~30분	프로젝트 폴더와 가상환경 생성	`.venv` 활성화
30~42분	PyTorch 및 Hugging Face 도구 설치	필요한 모듈 import
42~52분	기존 `check_env.py` 실행	GPU/CUDA/버전 정보 확인
52~60분	오류분류·재현정보 저장·평가	환경 점검표 제출

이번 차시 산출물

- `lab_llm/.venv` : 독립 실행 환경
- `lab_llm/docs/` : 패키지 버전·장비 확인 로그
- **환경 점검표**: Python, PyTorch, CUDA, GPU, 여유 저장공간, 모델 접근 가능 여부

12차시와의 연결

`환경 구축 → LLM 이해 → 모델 실행 → LoRA → 양자화 → 전후 비교 → 데이터 → SFTTrainer → SFT+LoRA → SFT+QLoRA → 병합/배포 → 종합평가`

2. 파인튜닝 환경이 중요한 이유

2-1. 프로그램 설치와 학습 성공은 다르다

생성형 AI 모델 학습에는 일반적인 Python 실행보다 많은 요소가 관여한다. 같은 코드라도 **GPU 드라이버, PyTorch 빌드, 패키지 의존성, 모델 접근 권한, 데이터 형식**에 따라 실행 가능 여부가 달라진다.

Python 코드 문법 검사 통과: 파이썬이 코드를 읽을 수 있다는 뜻이다.

라이브러리 `import` 성공: 모듈이 설치되었고 최소한의 로딩이 가능하다는 뜻이다.

`torch.cuda.is_available()` 참: PyTorch에서 CUDA 장치를 사용할 수 있다는 뜻이다.

모델 다운로드·추론 성공: 모델 형식과 메모리 조건을 통과했다는 뜻이다.

실제 학습 한 단계 성공: 순전파·손실 계산·역전파·가중치 갱신이 수행됐다는 뜻이다.

앞 단계가 성공해도 뒤 단계의 성공을 자동으로 보장하지 않는다.

2-2. 환경의 다섯 계층

계층	구성 요소	발생 가능한 문제
운영체제	Ubuntu, 파일 권한, 네트워크	저장공간 부족·권한 오류
연산장치	NVIDIA GPU, 드라이버	드라이버 미탐지·VRAM 부족
언어환경	Python 3.11, `.venv`, pip	파이썬 버전 충돌
학습도구	PyTorch, transformers, trl, peft	라이브러리 버전 불일치
실습자원	모델, 데이터셋, 결과 폴더	인증·형식·경로 오류

2-3. 실습 전 확인 질문

1. `python` 명령이 가리키는 실행 파일은 무엇인가?
2. 설치한 라이브러리가 현재 가상환경에 들어 있는가?
3. 운영체제가 GPU를 인식하는가? PyTorch도 인식하는가?
4. 실습용 모델과 데이터가 저장될 공간이 있는가?

판정 원칙: 모르는 값은 추측해서 기입하지 않고 실제 명령의 출력으로 확인한다.

3. 핵심 개념: LLM 학습에 쓰이는 메모리

3-1. 용어 연결

LLM(Large Language Model, 라지 랭귀지 모델·대규모 언어 모델): 대량의 텍스트를 학습해 토큰의 관계를 추론하는 모델이다.

Fine-Tuning(파인튜닝·추가 학습): 사전학습된 모델을 목적 데이터로 다시 조정하는 학습 방법이다.

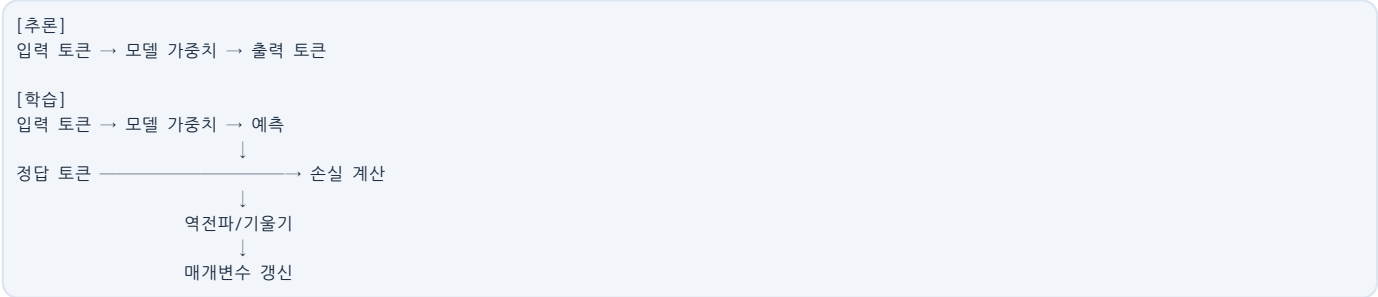
GPU(Graphics Processing Unit, 지피유·병렬 연산 장치): 행렬·벡터 계산을 병렬로 처리한다.

VRAM(Video RAM, 브이램·GPU 전용 메모리): GPU가 모델 가중치와 학습 계산 결과를 저장하는 공간이다.

CUDA(쿠다·엔비디아 GPU 병렬연산 플랫폼): CUDA 대응 GPU를 PyTorch에서 활용하는 연산 기반이다.

3-2. 왜 추론보다 학습이 무거울까?

추론은 일반적으로 입력을 받아 다음 토큰을 생성한다. 학습은 예측과 정답의 차이인 **손실(Loss, 로스·오차 값)**을 계산하고, 역전파로 미분값을 구한 뒤, 학습 가능한 매개변수를 갱신한다.



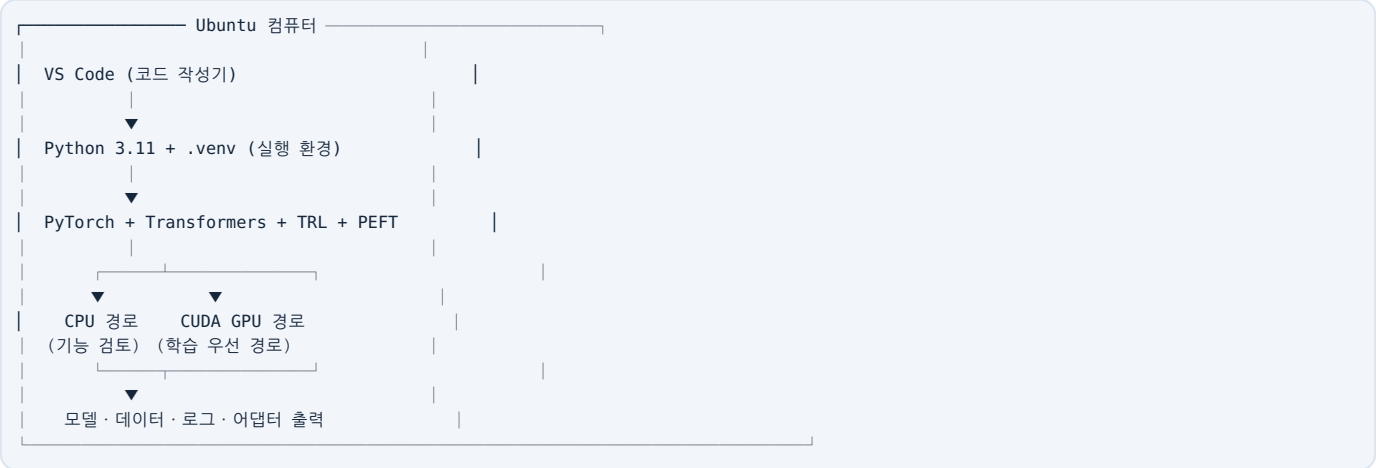
3-3. GPU 메모리를 구성하는 요소

요소	설명	일반적 증가 요인
모델 가중치	사전학습 매개변수	모델 규모, 저장 비트 수
활성값	계층 중간 계산 결과	배치 크기, 시퀀스 길이
기울기	학습 대상 매개변수의 미분값	학습 대상 가중치 수
옵티마이저 상태	가중치 갱신에 쓰이는 중간값	옵티마이저 유형

LoRA(로라·저랭크 적응)는 학습 대상 매개변수를 줄이고, **QLoRA(큐로라·양자화 기반 로라)**는 보통 기반 모델을 저비트로 적재해 메모리 요구량을 줄이는 방향이다. 두 기술의 원리는 4~5차시에서 자세히 다룬다.

4. 전체 실습환경 구조 및 파일 조직

4-1. 실행 구조



연결 관계: VS Code는 편집기이고, 계산은 Python과 라이브러리가 실행한다. GPU는 Python 코드를 실행하는 CPU를 대체하는 것이 아니라 대규모 텐서 계산을 가속한다.

4-2. 폴더 구조

```
lab_llm/
├── .venv/              # 독립 Python 가상환경
├── 실습코드/           # 공통 학습/검증 Python 코드
├── data/               # train.jsonl, eval.jsonl
├── models/             # 원본 모델 · LoRA 가중치
├── outputs/            # 출력 결과 및 비교 보고
└── docs/               # 환경 · 버전 기록 파일
```

4-3. 주의할 경계

`.venv` 는 장비 종속성이 있으므로 다른 컴퓨터로 복사해 재사용하지 않는다.

`models/` 에는 대용량 가중치가 생기므로 공개 Git 저장소에 무조건 올리지 않는다.

`data/` 에는 개인정보·라이선스 제한·비공개 데이터가 없는지 검토한다.

API 토큰이 필요한 경우 코드에 직접 넣거나 화면 캡처로 공개하지 않는다.

실습 확인

`pwd` 로 현재 경로를 확인하고, `mkdir -p models outputs docs` 로 결과용 폴더를 준비한다.

5. Ubuntu 사전 점검과 Python 준비

5-1. 시스템 상태 확인

터미널에서 아래 명령을 **한 줄씩** 실행한다. 결과의 실제 값은 장비마다 다르다.

```
# 운영체제 이름과 배포판 버전 확인
lsb_release -a

# 실제 Python 3.11 설치 여부 확인
python3.11 --version

# 저장공간 사용량과 여유 용량 확인
df -h .

# 메모리 총량과 사용량 확인
free -h

# NVIDIA GPU 및 드라이버 표시 (미설치 시 오류 가능)
nvidia-smi
```

5-2. 명령별 해석

`lsb_release -a` : 현재 운영체제와 버전을 표시한다. 명령 미설치 시 `cat /etc/os-release`를 사용한다.

`python3.11 --version` : `Python 3.11.x` 형태인지 확인한다. 출력이 없으면 별도 Python 설치 절차가 필요하다.

`df -h .` : 현재 폴더가 위치한 파티션의 남은 공간을 확인한다.

`free -h` : 시스템 RAM 사용량을 확인한다. VRAM 확인 명령과 혼동하지 않는다.

`nvidia-smi` : 드라이버가 보고하는 GPU 종류·상태·메모리 사용량을 확인한다.

5-3. 가상환경 준비에 필요한 기본 도구

```
# 패키지 목록 갱신 (시스템에 연결된 저장소 조회)
sudo apt update

# Python 3.11 가상환경 패키지와 Git 설치
# python3.11-venv가 현재 저장소에 없으면
# 사용 중인 Ubuntu 버전의 Python 설치 경로부터 확인
sudo apt install -y python3.11-venv git
```

실행 결과의 예: `Python 3.11.9`는 형식 예시이며, 이 교재 제작 환경에서 학습용 PC를 직접 검사해 확인한 값이 아니다.

실습 기록

운영체제: _____ / Python: _____ / GPU 모델: _____ / VRAM: _____ / 남은 저장공간: _____

6. 프로젝트와 `.venv` 생성

6-1. 작업 위치와 독립 가상환경

```
# 강의 실습용 프로젝트 폴더 생성 및 이동
mkdir -p ~/lab_llm
cd ~/lab_llm

# Python 3.11용 가상환경을 .venv 폴더에 생성
python3.11 -m venv .venv

# 현재 터미널에 가상환경 활성화
source .venv/bin/activate

# 실행 중인 Python 위치 · 버전 확인
which python
python --version

# pip 설치 도구 갱신
python -m pip install --upgrade pip
```

핵심 해설

`python3.11 -m venv .venv` 는 시스템 Python을 제거하거나 변경하지 않고 **프로젝트 전용 패키지 공간**을 생성한다. `source` 는 현재 셸의 환경변수를 변경해 이 가상환경의 `python` 을 우선 사용하게 한다.

6-2. 활성화 확인

```
# 프로젝트의 .venv 안에 python이 있는지 확인
which python

# pip도 동일한 Python에 묶여 있는지 확인
python -m pip --version

# 작업 종료 시 가상환경 비활성화
deactivate
```

출력 경로에 `lab_llm/.venv/bin/python` 이 포함되면 활성화 상태를 확인할 수 있다. `deactivate` 이후 다시 실습하려면 `source .venv/bin/activate` 를 실행한다.

6-3. VS Code와 연동

1. `code ~/lab_llm` 로 프로젝트 폴더를 연다.
2. Python 확장이 설치되어 있다면 명령 팔레트에서 **Python: Select Interpreter**를 선택한다.
3. `~/lab_llm/.venv/bin/python` 을 인터프리터로 지정한다.
4. 터미널과 VS Code의 선택한 인터프리터가 같은 경로인지 확인한다.

주의: GUI 편집기에서 실행하는 Python과 터미널의 Python이 서로 달라지는 문제를 피한다.

7. PyTorch와 Hugging Face 도구 설치

7-1. PyTorch 먼저 설치

PyTorch(파이토치·딥러닝 연산 프레임워크)는 **운영체제·패키지 도구·연산 플랫폼(CPU/CUDA)**에 맞는 공식 설치 명령을 선택한다.

공식 설치 선택기: https://pytorch.org/get-started/locally/

1. OS에 **Linux**, Package에 **Pip**, Language에 **Python**을 선택한다.
2. GPU 환경이면 실제 드라이버에 적합한 CUDA 빌드를 선택한다.
3. 표시된 공식 설치 명령을 **활성화된 `.venv`** 터미널에서 실행한다.
4. 이후 `python -c "import torch; print(torch.__version__)"` 로 로딩을 확인한다.

중요: 드라이버에 나타나는 CUDA 버전은 드라이버가 지원하는 기능 범위에 관한 정보이며, PyTorch 패키지가 실제로 사용하는 CUDA 런타임과 반드시 같지는 않다. 설치 명령을 추측해서 고정하지 않는다.

7-2. 학습도구 설치

```
# 가상환경 내부에서 실행: Hugging Face 관련 패키지
python -m pip install -U transformers datasets accelerate

# SFTTrainer(지도 파인튜닝 학습기)와 PEFT 설치
python -m pip install -U trl peft safetensors

# QLoRA 준비: 환경에 따라 지원 조건 확인 필요
python -m pip install -U bitsandbytes

# 핵심 모듈을 Python에서 찾을 수 있는지 검사
python -c "import torch, transformers, trl, peft; print('IMPORT_OK')"
```

패키지 역할

패키지	기능
`torch`	텐서 연산, 역전파, GPU 활용
`transformers`	모델·토크나이저 적재, 생성
`datasets`	학습 데이터의 로딩·처리
`accelerate`	장치 배치·학습 실행 지원
`trl`	`SFTTrainer` 제공
`peft`	LoRA와 같은 효율적 학습 기법 제공
`safetensors`	모델 텐서 저장 형식 지원
`bitsandbytes`	저비트 양자화 연산 기능

검증 수준: `IMPORT_OK` 만으로 GPU 학습 성공이 확인되는 것은 아니다.

8. 핵심 코드: 환경을 측정하는 `check_env.py`

이 개선판에 **독립 실습 파일** **`check_env.py`** **를 동봉**했다. 아래 코드는 해당 파일과 일치한다.

```
"""01차시: Python/PyTorch/CUDA 환경과 실습 가능 여부를 확인합니다."""
import importlib.metadata as md
import platform

print('Python:', platform.python_version())
for name in ('torch', 'transformers', 'trl', 'peft', 'datasets', 'accelerate', 'bitsandbytes'):
    try:
        print(f'{name}: {md.version(name)}')
    except md.PackageNotFoundError:
        print(f'{name}: 미설치')

try:
    import torch
    print('CUDA 사용 가능:', torch.cuda.is_available())
    if torch.cuda.is_available():
        print('GPU:', torch.cuda.get_device_name(0))
        print('총 VRAM(GiB):', round(torch.cuda.get_device_properties(0).total_memory / 2**30, 2))
        print('BF16 지원:', torch.cuda.is_bf16_supported())
except ImportError:
    print('PyTorch 미설치')
```

8-1. 코드 풀이 — 이 코드가 무엇을 하는가

`importlib.metadata.version(name)` : 설치된 **배포 패키지 버전**을 문자열로 읽는다. 해당 패키지가 없으면 예외가 발생한다.

`try` / `except` : 한 패키지가 없어도 전체 점검을 중지하지 않도록 한다.

`torch.cuda.is_available()` : PyTorch의 CUDA 사용 가능 여부를 참/거짓으로 표시한다.

`get_device_name(0)` : PyTorch가 인식한 첫 번째 CUDA GPU의 장치 이름을 표시한다.

`total_memory / 2**30` : 첫 번째 GPU의 전체 전용 메모리 용량을 GiB 단위 수치로 환산한다. 출력은 정확하게 **GiB** 단위로 표시한다.

`is_bf16_supported()` : 해당 환경에서 BF16 연산 지원 가능 여부를 조회한다.

8-2. 실행

```
# 01_환경구축 폴더에서 실행
python check_env.py
```

예시 출력(`CUDA 사용 가능: True`)은 형식 설명일 뿐, 사용자 GPU의 실측 결과를 대신하지 않는다.

9. 실습 검증·오류 분류·재현정보 저장

9-1. 5단계 검증 루프

```
1. Python 확인
     ↓ 성공
2. 라이브러리 import 확인
     ↓ 성공
3. nvidia-smi 드라이버 확인
     ↓ 성공
4. PyTorch CUDA 확인
     ↓ 성공
5. 환경 로그 기록 및 보관
```

첫 오류가 발생한 단계에서 멈추어 **현재 경로·실행 명령·전체 오류 메시지**를 기록한다. 문제 발생 시 처음부터 모든 패키지를 삭제하지 않는다.

9-2. 현상별 점검표

현상	우선 확인	대응
`python3.11: command not found`	Python 설치/버전	현재 Ubuntu 버전·설치 방법 조사
`.venv` 생성 실패	`python3.11-venv`	패키지 설치, 저장공간 확인
`ModuleNotFoundError`	`which python`, `pip show`	가상환경 내 패키지 재설치
`nvidia-smi` 실패	드라이버/GPU 연결	OS 수준 드라이버 문제부터 해결
CUDA는 False	PyTorch 빌드/드라이버	공식 선택기에서 적합 빌드 재설치
모델 다운로드 실패	네트워크·접근조건	HTTP 오류·사용약관·저장공간 확인
메모리 부족	VRAM 사용량	배치·시퀀스 축소, 다른 작업 종료

9-3. 버전·환경 기록

기존 패키지의 `실습코드/report_versions.sh`를 사용한다.

```
# 보고용 셸 스크립트 실행
bash 실습코드/report_versions.sh

# 실제 생성물 목록 확인
ls -lh docs/

# 가상환경 패키지 목록 중 핵심 항목 확인
grep -Ei '^(torch|transformers|trl|peft|datasets|accelerate)==' docs/pip_freeze.txt
```

스크립트는 `docs/python_version.txt`, `docs/pip_freeze.txt`, `docs/nvidia_smi.txt`, `docs/ollama_version.txt`를 남긴다. 시스템마다 드라이버·패키지 버전이 달라질 수 있으므로 로그를 **날짜와 장비별**로 보관한다.

추가 실습

환경 점검 파일을 새 터미널에서 한 번 더 실행하고 결과가 동일한지 비교한다. 프롬프트에 `.venv`가 보이는지 확인한다.

10. 실습 평가 및 다음 차시 준비

10-1. 최종 체크리스트

[ ] 원본 과정안내서의 교육 순서와 본 차시의 역할을 설명할 수 있다.

[ ] Python 3.11 실행 경로와 버전을 확인했다.

[ ] 프로젝트의 `.venv`를 만들고 활성화했다.

[ ] PyTorch 및 Hugging Face 핵심 도구를 설치하거나 설치 차단 원인을 기록했다.

[ ] `nvidia-smi`와 PyTorch CUDA 결과를 구별해 기록했다.

[ ] GPU 이름과 전용 메모리 용량을 실제 출력에 근거해 적었다(해당 GPU가 있는 경우).

[ ] `check_env.py` 실행 결과를 저장했다.

[ ] `report_versions.sh`의 결과 파일이 생성됐는지 확인했다.

[ ] 모델·데이터·로그 저장 폴더를 준비했다.

10-2. 이해도 평가(5문항)

1. 시스템에서 Python 3.11이 설치되어 있어도 가상환경을 만드는 이유는?
2. `nvidia-smi`가 정상 출력되지만 `torch.cuda.is_available()`이 False인 경우, 어떤 계층을 확인해야 하는가?
3. VRAM이 부족한 상황에서 실습 조건을 줄이는 방법 2가지는?
4. `pip freeze`와 `nvidia-smi` 결과를 함께 저장하는 이유는?
5. 라이브러리 `import` 성공과 학습 성공은 왜 다른가?

평가 기준: 5문항 중 4문항 이상을 설명하고, 실제 환경 기록을 제출하면 완료. GPU가 없는 환경은 학습 가능 여부와 대체 경로를 명시하면 된다.

10-3. 제출물 템플릿

항목	실측 결과	판정
Python 버전		정상 / 수정 필요
`which python` 경로		정상 / 수정 필요
`torch` 버전		정상 / 수정 필요
CUDA 사용 가능		True / False
GPU·VRAM		확인 / 미확인
설치 패키지 확인		완료 / 미완료
환경 기록 파일		생성 / 미생성

다음 차시: 02차시에서는 사전학습 모델의 구조, 토크나이저·임베딩·트랜스포머 흐름, 모델 규모와 실습 하드웨어 선택의 관계를 학습한다.

참고자료(2026-11-04 확인)

PyTorch 공식 설치 안내: https://pytorch.org/get-started/locally/

Hugging Face TRL SFTTrainer 문서: https://huggingface.co/docs/trl/sft_trainer

Hugging Face PEFT 공식 Quicktour: https://huggingface.co/docs/peft/quicktour

동봉 `check_env.py`, `report_versions.sh`

기술 검증 범위: 본 자료의 명령과 Python 스크립트는 문법·정적 검증을 수행할 수 있다. 실제 Ubuntu 설치, GPU 학습, 모델 다운로드와 가상 환경 실행 결과는 수강 환경에서 수행·기록해야 한다.

적대적 검증·개선 기록 (2026-11-05)

반영: 기존 코드·교안의 핵심 구조를 보존했습니다. 실제 GPU 실험은 별도 실행 검증이 필요합니다.

검증 구분: 코드 구문·입력 데이터·PDF 및 HTML 생성과 검증은 수행했으나 실제 모델 다운로드·GPU 학습·추론 품질·Ollama 배포는 별도의 실물 검증이 필요합니다.

재현성: 교육용 5스텝 학습의 손실 감소나 정확도 향상을 보장하지 않습니다. 모델 ID, 데이터 버전, 장치·패키지 버전, 난수 시드, 생성 설정을 기록해야 합니다.

공식 근거: SFTTrainer, PEFT 연동, 템플릿, PEFT 모델 저장, 양자화, Ollama GGUF (2026-11-05 확인)